

data will be stored. The *defn* form is used to define a function. The first parameter is the name of the function to be defined, the second is a vector containing the parameters, and the rest is the actual code of the function.

```
(import sqlite3)
(import sys)

(defn get-connection []
  (.connect sqlite3 "register.db"))
```

If we write the same code in Python, it would read as shown below. Notice how similar the code is to the Hy version of it. While we need to have a return statement in Python code in order to return a value to the caller, this is not needed in Hy. The value of the last expression evaluated is automatically returned.

```
import sqlite3
import sys

def get_connection():
    return sqlite3.connect("register.db")
```

Now that we have a function to connect to a database, let's write a function to create a database table as shown in the following example. The function takes a single parameter connection and uses it to execute the *sql* statement in the SQLite database. The last step is to return the connection:

```
(defn create-schema [connection]
  (.execute connection "create table if not exists person
(name text not null, phone text)")
  connection)
```

If the code had been written in Python, it could read as follows:

```
def create_schema(connection):
    connection.execute("create table if not exists person
(name text not null, phone text)")
    return connection
```

Manipulating the data

There are several ways in which to manipulate our data: create, read, update and delete. Let's write a function for each of these operations, starting from creating a new person. Because our person has only very few attributes and no methods, we do not create a class for this person. Instead, we'll pass around dictionaries and use keywords as keys. This is easier and requires less code than creating a class. So whenever you see code like `('(:name person)')` you know that we are getting the `:name` value from a person dictionary.

```
(defn insert-person [connection person]
  (let [[params (:name person) (:number person)]]
    (try (with [connection] (.execute connection "insert into
person (name, phone) values (?, ?)" params))
      (catch [e Exception] (print e)
        (print "failed to add a person")))))
```

This function takes two parameters: the connection to the database and the dictionary containing values of the person. Let's use the 'let' form to bind a *tuple* (name and number) into the *params* variable. You could think of it as a local variable, since it is accessible only within the let form and ceases to exist when the execution flow leaves the form. The final step is to instruct the connection to execute the *sql*-statement and supply parameters to it.

Error handling is done with try-catch block. If the code inside try-block throws an exception, the execution immediately skips to catch-block where the error is reported. The with-form takes advantage of the context manager provided by the SQLite connection. When execution enters the with-block, a database transaction is automatically started. If no exceptions occur, the transaction is committed when the with-block ends and changes are saved into the database. However, if there is an exception, the transaction is automatically rolled back and execution continues from the catch-block.

After the data has been saved, it would be nice to be able to load it too. Let's write three functions for this. One is used to load a single person by ID. The second function is a general search that can load multiple persons in a single query. The third is a function that turns a database row into a person dictionary:

```
(defn load-person [connection person-id]
  (row-to-person (.fetchone (.execute connection "select OID,
name, phone from person where OID=?" (: person-id)))))
```

```
(defn query-person [connection search-criteria]
  (let [[search-term (+ "%" search-criteria "%")]
        [search-param (: search-term search-term)]]
    (list-comp (row-to-person row)
      [row (.fetchall (.execute connection "select
OID, name, phone from person where name like ? or phone like ?"
search-param))])))
```

```
(defn row-to-person [row]
  (:id (get row 0) :name (get row 1) :number (get row 2)))
```

The query-person function introduces a new construct '*list-comp*', which is used to perform list comprehensions. In the example, we are executing a *sql* query and fetching all matching records from the database. After that, we iterate over each row and call the row-to-person function for each of them. The results are collected together and returned as a list of dictionaries containing persons.

In row-to-person, a single database row is transformed into a dictionary. We are using keywords (:id, :name and :number) as